

EC9110

MODERN MOBILE COMMUNICATION

ASSIGNMENT – END-TO-END 5G SIMULATION

MEMBER A 2022/E/XXX
MEMBER B 2022/E/XXX
MEMBER C 2022/E/XXX
MEMBER D 2022/E/XXX
MEMBER E 2022/E/XXX
MEMBER F 2022/E/XXX
MEMBER G 2022/E/XXX
MEMBER H 2022/E/XXX

SEMESTER 07

07 OCT 2026

CONTENTS

1	Service scenario and requirements	2
2	Slice design and parameter rationale	2
3	End-to-end architecture	3
4	Implementation by component	4
4.1	Core network – control plane (Member A (2022/E/XXX))	5
4.2	Core network – user plane (Member B (2022/E/XXX))	5
4.3	Radio access network (Member C (2022/E/XXX))	5
4.4	User equipment and provisioning (Member D (2022/E/XXX))	5
4.5	Traffic generation (Member E (2022/E/XXX))	6
4.6	Signalling capture and analysis (Member F (2022/E/XXX))	6
4.7	Sandbox application (Member G (2022/E/XXX))	6
4.8	Platform and integration (Member H (2022/E/XXX))	6
5	Sandbox application	7
6	Experimental method	8
7	Results	8
7.1	E1 – Deployment and UE-to-data-network connectivity	8
7.2	E2 – Signalling: registration, session establishment and slice selection	9
7.3	E6 – Temporal traffic profiles per slice	11
7.4	E3 – Isolation of URLLC and mMTC from an eMBB surge	11
7.5	E4 – Live slice-parameter changes	12
7.6	E5 – Dedicated vs. shared user plane for URLLC	13
8	Analysis: from scenario to measured result	14
9	Limitations of a software-only environment	15
10	Challenges and resolutions	16
11	Conclusions and future work	17

1 SERVICE SCENARIO AND REQUIREMENTS

Scenario. A manufacturer runs a private 5G SA network on its factory campus. Three service classes share the same radio cell and core, but their requirements are almost opposite [1, 2]:

1. **Motion control / AGV coordination (URLLC).** A PLC exchanges small state/command packets with robot arms and automated guided vehicles every 10 ms (100 Hz, 64-byte payloads). 3GPP TS 22.104 places this class of cyber-physical control in the low-millisecond end-to-end latency range, with very high availability [3]. Throughput is tiny (about 0.05 Mbit/s); what matters is the latency *tail* (p99) and isolation from other traffic.
2. **Visual inspection and AR maintenance (eMBB).** Inspection cameras and AR headsets pull high-rate video segments from an edge server. The traffic is bursty: segment downloads (“on”, 20 s) alternate with idle buffer periods (“off”, 5 s). Tens to hundreds of Mbit/s per slice are needed, with no strict latency bound. The slice must be *capped* so that it cannot starve the other slices.
3. **Environmental and asset telemetry (mIoT).** Hundreds of sensors behind IoT gateways send about 100-byte JSON reports. Arrivals follow a Poisson process (mean period 5–10 s per device). The requirements are connection density and delivery reliability. Latency below one second is acceptable, and the aggregate rate is a few kbit/s.

Derived requirements. Table 1 turns the scenario into the testable targets used throughout the report. The same targets drive the slice design (§2), the traffic generators (§4.5) and the pass/fail reading of the results (§8).

Table 1: Per-slice service requirements derived from the scenario (targets for a software testbed; the absolute values are bounded by §9).

	eMBB (SST 1)	URLLC (SST 2)	mIoT (SST 3)
Traffic model	TCP downlink, 2–4 flows/UE, 20 s on / 5 s off	UDP 64 B request/echo, 100–200 Hz	UDP ≈100 B reports, Poisson, 200–3000 devices
Key KPI	throughput, capped at slice MBR	RTT p50/p99, jitter, loss	delivery ratio, ACK latency
Target	≥100 Mbit/s aggregate, ≤ MBR	RTT p99 < 10 ms, loss < 0.1 %	loss < 1 %, ACK < 1 s
Isolation need	must not affect others	must be unaffected by eMBB load	best effort

2 SLICE DESIGN AND PARAMETER RATIONALE

Each service class maps to one S-NSSAI. We use the standardised SST values (1 = eMBB, 2 = URLLC, 3 = mIoT [4]). We add an explicit SD so that further tenants can later be added under the same SST. Each S-NSSAI also gets its own DNN and IPv4 pool, so a packet’s source address alone identifies its slice. That makes per-slice policing, capture filtering and KPI attribution trivial. Table 2 lists every parameter; the reasons for each choice follow.

Table 2: Slice configuration (configs/slices.yaml, configs/open5gs/*.yaml, scripts/30_subscribers.sh).

Parameter	eMBB	URLLC	mIoT
S-NSSAI (SST/SD)	1 / 000001	2 / 000002	3 / 000003
DNN / UE pool	internet / 10.45.0.0/16	urllc / 10.46.0.0/16	iot / 10.47.0.0/16
User plane	UPF-A (shared), ogstun	UPF-B (dedicated), ogstun2	UPF-A (shared), ogstun3
N3 endpoint	10.10.0.1	10.10.0.3	10.10.0.1
5QI / ARP priority	9 / 8	80 / 1	9 / 12
Session-AMBR DL/UL	200 / 50 Mbit/s	20 / 20 Mbit/s	2 / 2 Mbit/s
Slice MBR (tc HTB)	150 Mbit/s	20 Mbit/s	5 Mbit/s
Added delay / loss	0 ms / 0 % (live-tunable)	0 ms / 0 % (live-tunable)	0 ms / 0 % (live-tunable)
Subscribed UEs	embb1, embb2, multi	urllc1, multi	miot1, miot2

Dedicated vs. shared user plane. The URLLC slice gets its own UPF process (UPF-B) with its own PFCP association, GTP-U endpoint and tun device. The aim is to isolate its forwarding path from the bursty eMBB load, both in CPU time and in queues. In a real deployment the same choice lets URLLC be placed at the factory edge (local breakout) while eMBB/mIoT can be centralised. mIoT carries only kbit/s, so a dedicated UPF would cost resources for no benefit; it therefore shares UPF-A with eMBB. Experiment E5 (§7.6) tests this decision directly by moving URLLC onto UPF-A under eMBB overload.

QoS parameters. 5QI 9 is the default non-GBR class for buffered video and TCP traffic, which fits eMBB and telemetry. URLLC uses 5QI 80. That is the standardised *non-GBR low-latency* class (10 ms packet delay budget), and it needs no GBR flow template. Delay-critical GBR 5QIs (82–85) would require PCC rules with GBR values, which UERANSIM’s default QoS-flow handling does not exercise. The ARP gives URLLC the highest priority (1) and mIoT the lowest (12). Session-AMBRs are provisioned in the UDR and delivered to the UE in the PDU Session Establishment Accept (verified with `nr-cli ps-list`, §7.1).

Why an extra tc layer. Open5GS’s UPF polices the Session-AMBR. We measured 230/57 Mbit/s for a 200/50 eMBB AMBR and 1.9 Mbit/s for the 2 Mbit/s mIoT AMBR with the tc caps lifted (`results/ambr_check.txt`). However, the AMBR is *per session*, is fixed when the session is established, and cannot express delay or loss. A software RAN also has no scheduler that could give one slice priority over another. We therefore implement a *per-slice* MBR and the delay/loss “transport” parameters with Linux tc: an HTB class plus a netem child per slice, in both directions. The downlink is shaped on the slice’s UPF tun device and the uplink on the N6 veth, with one class per UE pool. Every parameter can be changed live (`tc class/qdisc change`) without touching any UE session. This is the “live slice parameter” exposed by the sandbox. The binding limit is the lower of the two: the tc MBR for eMBB (150 Mbit/s) and URLLC (20 Mbit/s), and the Session-AMBR for mIoT (2 Mbit/s). The values cover the scenario’s peak demand (inspection video $\approx 2 \times 50$ Mbit/s plus headroom; URLLC $\approx 100 \times$ its nominal 0.05 Mbit/s; mIoT far above its few kbit/s), while staying low enough to protect the other slices.

Slice selection path. The gNB advertises all three S-NSSAIs in NG Setup. The AMF supports them in its PLMN list. The NSSF maps each S-NSSAI to an NSI served by the NRF. The SMF registers `info.s_nssai` for all three. Each subscriber’s UDR record lists only its permitted S-NSSAIs, so a UE that requests an unsubscribed slice is rejected (§7.1).

3 END-TO-END ARCHITECTURE

Figure 1 shows the deployed system. All 5G software needs Linux kernel features (SCTP, TUN, network namespaces, tc), so the testbed runs inside one Ubuntu 24.04 environment with 4 vCPU and 6 GB. The host operating system only provides that environment. Two hosts are supported (Table 3), and everything from the provisioning script upwards is identical on both. **All results in this report were measured on the macOS host** (Apple M4, 16 GB) running Lima [5] with Apple’s Virtualization framework (Ubuntu arm64, kernel 6.8). On Windows, WSL2 [6] provides the same Ubuntu release in the PC’s own architecture (normally amd64).

Table 3: Supported host platforms. Only the layer below Ubuntu differs.

	macOS (used for all results)	Windows 10/11
Virtualisation	Lima 2.2.1, <code>vz</code> (Apple Virtualization.framework)	WSL2 (Hyper-V lightweight VM)
Linux guest	Ubuntu 24.04, arm64, kernel 6.8	Ubuntu 24.04, amd64 (arm64 on Windows on ARM), Microsoft WSL kernel
Resources	<code>vm/lima-5g.yaml</code> : 4 vCPU, 6 GiB	<code>vm/wslconfig.example</code> : 4 vCPU, 6 GB
Repository at	virtiofs mount of the Mac folder	symlink to a clone inside the WSL filesystem
<code>/e2e5g</code>		
Preparation	<code>limactl create/start</code>	<code>scripts/wsl_setup.sh</code> (systemd, path, line endings, live kernel checks)
Sandbox URL	<code>http://localhost:8501</code> (Lima port forward)	<code>http://localhost:8501</code> (WSL localhost forwarding)
Removal	<code>scripts/uninstall_mac.sh</code>	<code>wsl --unregister Ubuntu-24.04</code>

Inside the Linux environment, three network namespaces model the three administrative domains:

- **ran**: the UERANSIM gNB and all UEs. Each UE PDU session appears as a `uesimtunN` interface. Because UEs live in their own namespace, downlink packets addressed to a UE cannot be short-circuited by the core's kernel; they *must* pass UPF → GTP-U → gNB.
- **root (core)**: the Open5GS NFs on loopback SBI addresses (HTTP/2, port 7777, through the SCP), MongoDB, and the two UPFs. A veth pair (`n3-core/n3-ran`, 10.10.0.0/24) carries both N2 and N3.
- **dn**: the external data network (192.168.100.2), reached over the N6 veth. It hosts the iperf3 servers, the URLLC echo server and the IoT collector. A NAT rule also gives the UE pools real Internet access.

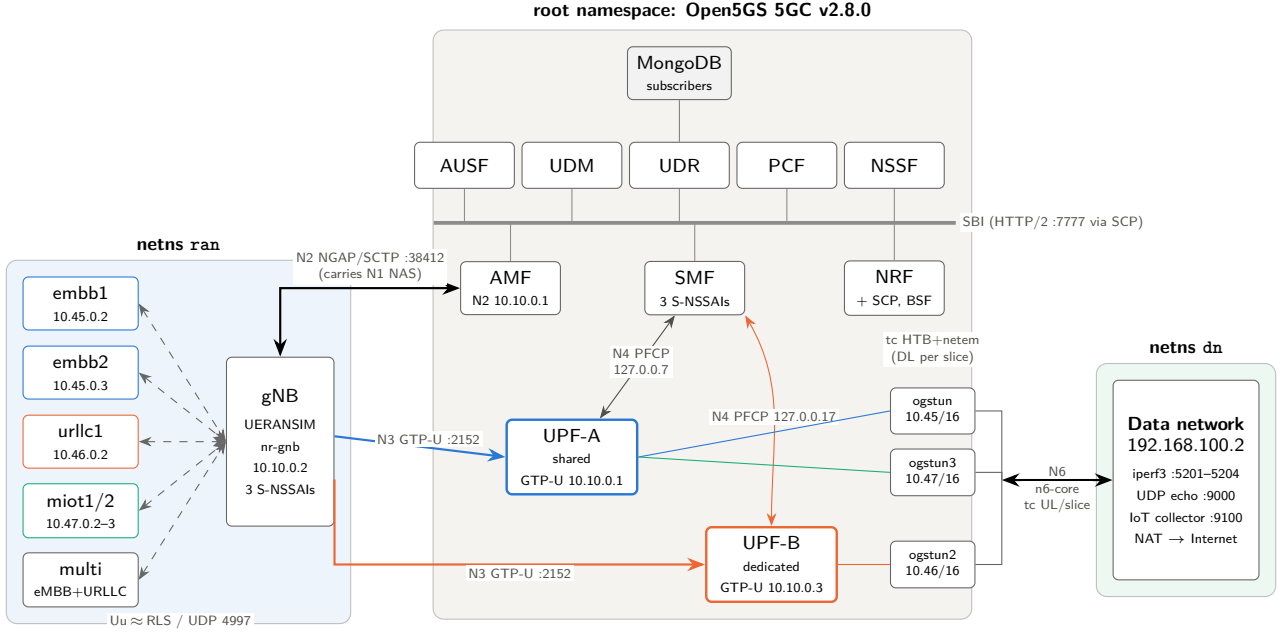


Figure 1: Deployed end-to-end architecture with all interfaces labelled. The colour of each user-plane path matches the slice colour used in all plots (eMBB blue, URLLC orange, mIoT green). Everything runs inside one Ubuntu 24.04 environment on the host (Lima on macOS, WSL2 on Windows).

Interfaces. **Uu** is replaced by UERANSIM's *Radio Link Simulation* (RLS): RRC/NAS and user data travel between `nr-ue` and `nr-gnb` as UDP datagrams on port 4997. That is the channel-simulation mode used in place of radio hardware. It has no PHY, no MAC scheduling and no fading; §9 covers what that implies. **N1** (NAS-5GS) rides inside NGAP. **N2** is NGAP over SCTP (gNB 10.10.0.2 → AMF 10.10.0.1:38412). **N3** is GTP-U over UDP 2152, terminating at 10.10.0.1 (UPF-A) or 10.10.0.3 (UPF-B). The SMF chooses the UPF per PDU session, and the choice reaches the gNB as the UL tunnel address in `PDU Session Resource Setup Request`. **N4** is PFCP between the SMF (127.0.0.4) and each UPF. **N6** is the veth to the `dn` namespace. The service-based interfaces (**N8**, **N10**, **N11**, **N12**, **N13**, **N15**, **N22**, `Nnrf`, `Nbsf`) are HTTP/2 on port 7777, using indirect communication through the SCP.

End-to-end path of one URLLC packet. The `urlc_probe` sends from 10.46.0.2 on `uesimtun2` in `ran`. `nr-ue` carries it over RLS to `nr-gnb`, which encapsulates it in GTP-U (TEID from the PDU session) towards 10.10.0.3. UPF-B decapsulates and writes it to `ogstun2`; the kernel then routes it over `n6-core` (tc class 1:20) into `dn`, where the echo server replies. The reply follows the same path in reverse: `ogstun2` egress (tc HTB/netem) → UPF-B → GTP-U → gNB → RLS → UE.

4 IMPLEMENTATION BY COMPONENT

Each component has a named owner who configured it and validated it on its own before integration (Table 4). All configuration, scripts, sandbox and analysis code are in the shared repository, <https://example.com/shared/End-to-End-5G-Simulation>, whose `README.md` gives the step-by-step setup for both platforms. The whole testbed is rebuilt from a clean Mac (Lima) or Windows PC (WSL2) with the same five scripts.

Table 4: Component ownership, deliverables and individual validation evidence.

Component	Owner	Main artefacts	Individual validation
Core – control plane	Member A (2022/E/XXX)	<code>amf</code> , <code>smf</code> , <code>nssf</code> , <code>nrf</code> , <code>ausf</code> , <code>udm</code> , <code>udr</code> , <code>pcf</code> , <code>bsf</code> , <code>scp</code> .yaml; <code>20_core.sh</code>	all NFs register at NRF; NG Setup accepted; Allowed NSSAI in Registration Accept
Core – user plane	Member B (2022/E/XXX)	<code>upf-a.yaml</code> , <code>upf-b.yaml</code> , SMF UPF selection, <code>10_netns.sh</code> (N3/N6, tun, NAT)	PFCP association with both UPFs; PFCP Session Establishment to the right UPF per DNN
Radio access network	Member C (2022/E/XXX)	<code>gnb.yaml</code> , RAN namespace, <code>40_ran.sh</code>	SCTP association + NGSetupResponse; per-UE PDU Session Resource Setup
UE & provisioning	Member D (2022/E/XXX)	<code>gen_ues.sh</code> (7 UE configs), <code>30_subscribers.sh</code> , <code>ue_cfg.py</code> , <code>ue_table.sh</code>	6 UEs RM-REGISTERED with 7 PDU sessions; negative UE rejected
Traffic generation	Member E (2022/E/XXX)	<code>embb.py</code> , <code>urllc_probe.py</code> , <code>miot_client.py</code> , DN servers	per-slice KPI CSVs; on/off, periodic and Poisson profiles verified (Fig. 5)
Signalling analysis	Member F (2022/E/XXX)	<code>50_capture.sh</code> , <code>analysis/analyze.py</code>	decoded call flow, timing and SBI tables (§7.2)
Sandbox application	Member G (2022/E/XXX)	<code>sandbox/app.py</code> , <code>testbed.py</code> , <code>slice_ctl.sh</code>	every control exercised from the UI (§5)
Platform & integration	Member H (2022/E/XXX)	<code>vm/lima-5g.yaml</code> , <code>vm/wslconfig.example</code> , <code>wsl_setup.sh</code> , <code>00_provision.sh</code> , <code>experiments/run_all.py</code> , <code>versions.sh</code>	clean rebuild; scripted E1–E7; Windows checks

4.1 Core network – control plane (Member A (2022/E/XXX))

Open5GS 2.8.0 [7] was installed from the project’s PPA (arm64 build on the Mac; the same PPA ships amd64 for Windows PCs). The NFs are not run as the packaged systemd units. `20_core.sh` starts them in dependency order (NRF → SCP → AUSF/UDM/UDR/PCF/NSSF/BSF → UPFs → SMF → AMF) from the repository configs, so that every parameter is under version control. The configs deviate from the defaults as follows. The PLMN is changed to the test PLMN 001/01. The AMF binds NGAP to 10.10.0.1 and supports three S-NSSAIs. Its integrity order is NIA2>NIA1 and its ciphering order is *NEA0 first*, so NAS stays readable in captures (the integrity protection is still checked). The NSSF has one NSI per S-NSSAI, and the SMF advertises `info.s_nssai` for all three slices. MongoDB 8.0 holds the subscriber records.

4.2 Core network – user plane (Member B (2022/E/XXX))

Two `open5gs-upfd` instances run with separate PFCP (127.0.0.7 / 127.0.0.17), GTP-U (10.10.0.1 / 10.10.0.3) and metrics addresses. The SMF selects the UPF by DNN (`pcf.client.upf[].dnn`). A second SMF/UPF-A configuration pair (`smf-shared.yaml`, `upf-a-shared.yaml`) puts URLLC on UPF-A for the shared-UPF experiment. Pool addressing lives in both the SMF and the UPF `session` lists, with an explicit `dev` so that each DNN uses its own tun device. `10_netns.sh` creates the tun devices persistently, so the `tc qdiscs` survive UPF restarts. It also sets the DN return routes and the NAT.

4.3 Radio access network (Member C (2022/E/XXX))

UERANSIM v3.2.7 [8] was built from source (gcc 13.3, CMake 3.28). The gNB (`nci 0x10`, TAC 1, 32-bit gNB-ID) runs in the `ran` namespace. It binds its RLS side to 127.0.0.1 and its N2/N3 side to 10.10.0.2. It lists all three S-NSSAIs; without this, the AMF would reject NG Setup for unsupported slices. `ignoreStreamIds` works around Open5GS’s SCTP stream allocation.

4.4 User equipment and provisioning (Member D (2022/E/XXX))

`gen_ues.sh` writes one UE config per subscriber from a single template, which avoids copy-paste drift. Each config holds the SUPI, K, OPc, null-scheme SUCI, the sessions (DNN + S-NSSAI), the configured/default NSSAI and the algorithms. `30_subscribers.sh` creates the matching UDR records with `open5gs-dbctl` and then

writes the per-slice QoS profile (5QI, ARP, Session-AMBR) into each session with `mongosh`. Seven subscribers exist: two eMBB, one URLLC, two mMTC gateways, one *multi-slice* UE subscribed to eMBB+URLLC (it holds two concurrent PDU sessions on two UPFs), and one negative-test UE subscribed only to eMBB that requests URLLC. At start-up `ue_cfg.py` derives each UE’s runtime config, keeping only the sessions of slices that are administratively up (§5).

4.5 Traffic generation (Member E (2022/E/XXX))

Each service class has a generator that matches its temporal profile. All generators write one KPI row per second (`traffic/kpi.py`) and bind to the UE’s PDU-session address, so all traffic enters the 5G user plane.

- **eMBB** (`embb.py`): wraps `iperf3` 3.16 [9] in reverse (downlink) mode with N parallel TCP streams in an on/off cycle (default 20s/5s). It parses the per-second rate and logs explicit zeros during off periods.
- **URLLC** (`urllc_probe.py`): sends sequence-numbered, timestamped 64-byte UDP packets at a fixed rate (100 Hz by default) to the DN echo server. It uses `select()` so that echoes are timestamped on arrival, and reports per-second RTT p50, p99, jitter (mean $|\Delta\text{RTT}|$) and loss (no echo within 1 s).
- **mMTC** (`miot_client.py`): simulates D sensors behind one gateway UE. Each sends a JSON report with exponential inter-arrival times (mean P); the collector returns a 16-byte ACK. It reports reports/s, ACK latency and loss (no ACK within 2 s).

4.6 Signalling capture and analysis (Member F (2022/E/XXX))

`50_capture.sh` runs `tshark` on `n3-core` (N2, N3) and `1o` (SBI, N4). The capture filter is SCTP, UDP 2152/8805 and TCP 7777. `analysis/analyze.py` decodes it with `nas-5gs.null_decipher` and `-d tcp.port==7777,http2`. It extracts per-UE procedure timing (keyed on RAN-UE-NGAP-ID and SUCI MSIN), PFCP sessions per UPF and SBI operations. It also draws the message ladder and every KPI figure from the CSVs.

4.7 Sandbox application (Member G (2022/E/XXX))

The sandbox has three layers, so the user interface never touches the testbed directly:

- **UI** (`sandbox/app.py`, Streamlit). It renders the slice cards, the live parameter forms, the KPI charts, the UE/PDU-session table and the event log. The KPI panel is a fragment that re-runs every 2 s, so only the charts refresh while the rest of the page stays still.
- **Control library** (`sandbox/testbed.py`). It exposes the actions: apply a scenario, slice up/down, start/stop traffic, set QoS, read KPIs and export a run. It starts traffic generators in the `ran` namespace as separate process groups bound to each UE’s current PDU address, so “stop” always removes every child process. Every action is appended to the run’s `events.csv`.
- **Slice control** (`scripts/slice_ctl.sh`). It applies the `tc` HTB/netem changes and the deregister/re-attach cycle for slice up/down. Administrative state lives in files under `/run/e2e5g`, so a page reload or a second browser sees the same state.

The experiment runner uses the same library, so every sandbox action can be reproduced by script. Section 5 shows the resulting interface.

4.8 Platform and integration (Member H (2022/E/XXX))

Every script assumes the repository at `/e2e5g` and an Ubuntu 24.04 environment, so the host only has to provide those two things. On macOS, `vm/lima-5g.yaml` defines the VM, mounts the repository at `/e2e5g` (virtiofs) and forwards port 8501 (sandbox) to the Mac. On Windows, WSL2 runs Ubuntu 24.04 with the resources in `vm/wslconfig.example`, and `scripts/wsl_setup.sh` prepares it. The script enables systemd, links the clone to `/e2e5g` and rejects CRLF line endings. It also live-tests every kernel feature the testbed needs (SCTP sockets, netns, veth, TUN, `tc` HTB and netem) before anything is installed. `00_provision.sh` is idempotent, detects the CPU architecture (arm64 or amd64) and installs every dependency on either platform. `experiments/run_all.py` restarts the testbed into a known state before each experiment and drives the same `testbed.py` API that the sandbox uses, so demo and measurements share one code path. `versions.sh` records the exact version of every component in `results/versions.txt` (Open5GS 2.8.0, MongoDB 8.0.32, UERANSIM v3.2.7, `iperf3` 3.16, TShark 4.2.2, `iproute2` 6.1.0, Python 3.12.3, Streamlit 1.65.0; guest Ubuntu 24.04.5 with kernel 6.8.0; host macOS 27.0 with Lima 2.2.1).

5 SANDBOX APPLICATION

The sandbox (`sandbox/app.py`, Streamlit 1.65 [10]) is the single control surface for someone who did not build the system. It runs as root inside the VM and is opened in a browser on the host (Mac or Windows) at `http://localhost:8501`. Its controls map one-to-one onto the required capabilities (Table 5). Every action goes through `sandbox/testbed.py`. That is the same library the scripted experiments use, so a result seen in the demo can be reproduced exactly by script. Every action is also appended to the run’s `events.csv`, which later annotates the plots.

Table 5: Sandbox capabilities and how each is realised in the testbed.

Requirement	UI control	Mechanism
Select a scenario	sidebar: <i>Service scenario</i> + <i>Apply</i>	preset of per-slice tc parameters + traffic profiles (S1 baseline, S2 factory peak, S3 degraded transport, S4 massive-IoT burst)
Bring slices up/down	per-slice <i>Slice up</i> / <i>Slice down</i>	switch-off deregistration of the slice’s UEs, then re-attach with a runtime config without (or with) that S-NSSAI; multi-slice UEs keep their other session
Start/stop traffic per slice	<i>Start</i> / <i>Stop traffic</i>	generator processes in the <code>ran</code> netns bound to the UE’s current PDU address (process groups, so stop is clean)
Change a slice parameter live	<i>Live slice parameters</i> form (MBR, delay, loss)	<code>tc class change</code> (HTB rate) and <code>tc qdisc change</code> (netem) on DL tun + UL N6 class; no session impact
View live per-slice KPIs	metrics + charts, auto-refresh 2 s	1-s KPI rows aggregated per slice: throughput, RTT p50/p99, loss
Export run data	<i>Export run (ZIP)</i>	KPI CSVs, event log, UE/PDU table, current QoS

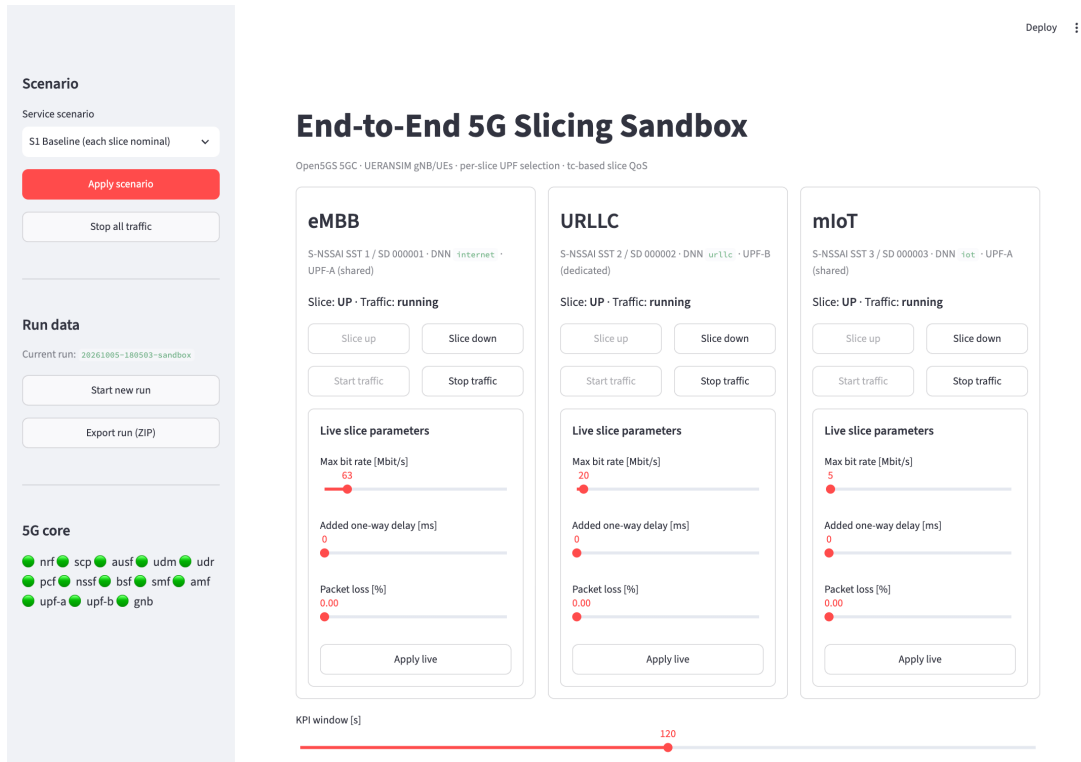


Figure 2: Sandbox during scenario S1. Each slice card shows its S-NSSAI, DNN and UPF, its admin and traffic state, and the live parameter form. The 5GC NF health is shown in the sidebar.

Design decision: slice down. Our first implementation released the slice’s PDU sessions with `nr-cli ps-release`. UERANSIM, however, re-establishes every session listed in its config within ≈ 200 ms, so the slice came straight back (with new addresses). The final design treats “slice down” as an administrative

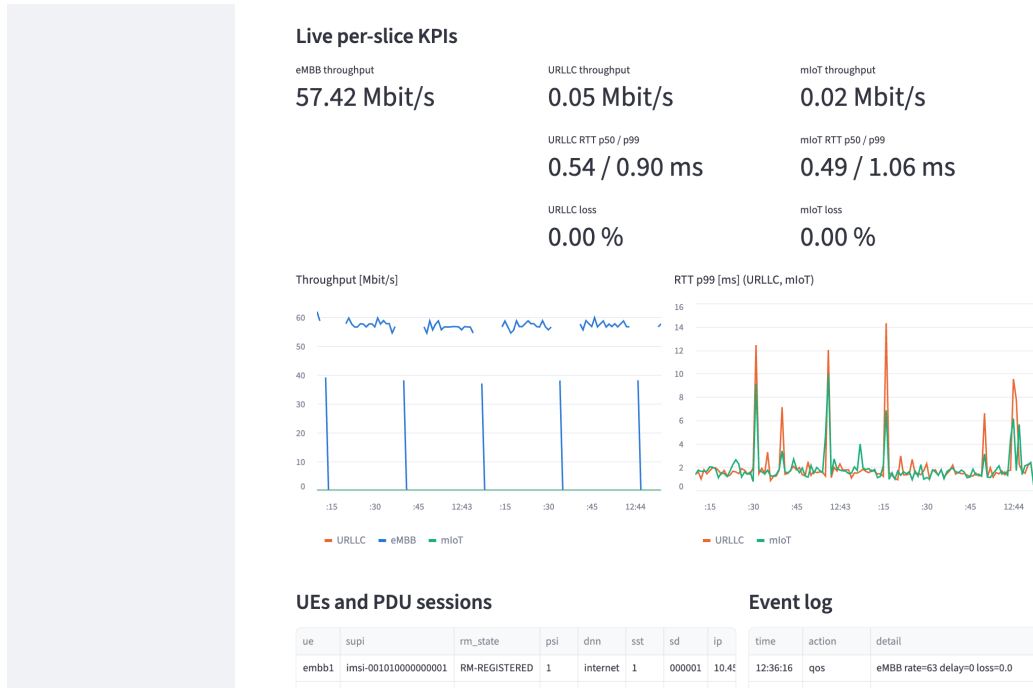


Figure 3: Sandbox live KPI panel, UE/PDU-session table and event log after a live change of the eMBB MBR.

state (`/run/e2e5g/state-<slice>`). The affected UEs perform a switch-off deregistration, which makes the AMF/SMF release all their contexts and PFCP sessions. They then re-attach from a runtime config filtered to the slices that are still up. This also exercises the deregistration and re-registration procedures on demand.

6 EXPERIMENTAL METHOD

Every experiment is scripted in `experiments/run_all.py` and starts from a known state: RAN stopped, core restarted, slice qdiscs reset to Table 2, and DN services restarted. The traffic generators log one KPI sample per second per UE. The analysis aggregates per slice and per second: throughput is summed over UEs, p50 is averaged, and p99 takes the worst UE. Phase boundaries come from the event log. The experiments follow one thread: *does each slice meet its Table 1 target on its own, under cross-slice load, after live re-parameterisation, and with or without a dedicated user plane?*

Metrics. RTT is measured end-to-end, UE↔DN, through RLS, gNB, GTP-U, UPF, tc and N6. One-way delays cannot be measured reliably because there is no shared clock across processes (§9). Throughput is the iperf3 receiver goodput at the UE. Loss is a missing echo/ACK after 1 s/2 s. Signalling latencies come from capture timestamps on `n3-core`.

7 RESULTS

All numbers below come from the runs stored in `results/` and were produced by `analysis/analyze.py`. Nothing was edited by hand.

7.1 E1 – Deployment and UE-to-data-network connectivity

All twelve 5GC processes and the gNB started; the NG Setup succeeded (gNB-N2 accepted[10.10.0.2], Number of gNBs is now 1), and the SMF formed PFCP associations with both UPFs. Listing 1 is the state reported by `nr-cli` after attach. Every UE is RM-REGISTERED. The multi-slice UE holds *two* PDU sessions in two pools (10.45.0.4 on eMBB, 10.46.0.3 on URLLC). The negative test UE `badslice` registers but obtains no address.

Listing 1: UE and PDU-session state after E1 attach (`results/e1/ue_table.txt`).

UE	SUPI	RM-STATE	PSI	DNN	SST	SD	ADDRESS	IFACE
badslice	imsi-001010000000007	RM-REGISTERED	1	urllc	2	000002	null	-
embb1	imsi-001010000000001	RM-REGISTERED	1	internet	1	000001	10.45.0.2	uesimtun0

Table 6: Experiment plan.

ID	Question	Procedure	Output
E1	Do all UEs register, get the right slice/IP/UPF, and reach the DN?	capture on; start gNB + 6 UEs + negative UE; ping (10×) from every PDU session; iperf3 5 s and HTTP per slice	UE table, pcap, ping/iperf logs
E2	Is registration, session establishment and slice selection visible in signalling?	decode the E1 pcap (NGAP/NAS/PFCP/SBI)	ladder, timing, SBI tables
E3	Is URLLC/mIoT isolated from an eMBB surge?	URLLC 100 Hz + mIoT 2×500 devices for 120 s; eMBB 2 UEs ×4 TCP streams in $t=30-90$ s	RTT/loss by phase
E4	Do live slice-parameter changes take effect, and only on that slice?	all slices active 120 s; $t=40$ s eMBB MBR 150→30 Mbit/s; $t=80$ s URLLC +5 ms each way	time series around changes
E5	Is a dedicated URLLC UPF justified?	2 runs: dedicated (UPF-B) vs shared (UPF-A); URLLC 200 Hz; eMBB cap removed, 2 UEs ×8 streams in $t=20-80$ s	RTT p99 series + CDF
E6	Do generators reproduce the temporal profiles?	scenario S1 for 90 s	per-slice profiles

embb2	imsi-0010100000000002	RM-REGISTERED	1	internet	1	000001	10.45.0.3	uesimtun1
miot1	imsi-0010100000000004	RM-REGISTERED	1	iot	3	000003	10.47.0.2	uesimtun3
miot2	imsi-0010100000000005	RM-REGISTERED	1	iot	3	000003	10.47.0.3	uesimtun4
multi	imsi-0010100000000006	RM-REGISTERED	1	internet	1	000001	10.45.0.4	uesimtun5
multi	imsi-0010100000000006	RM-REGISTERED	2	urllc	2	000002	10.46.0.3	uesimtun6
urllc1	imsi-0010100000000003	RM-REGISTERED	1	urllc	2	000002	10.46.0.2	uesimtun2

The AMF rejects the unsubscribed slice explicitly, and the UE reports the corresponding 5GMM cause:

```
[amf] [gmm] WARNING: [imsi-0010100000000007] Ue requested DNN "urllc" Not Supported OR Not Subscribed in the Slice
[ue ] [nas] [error] MM status received with cause [DNN_NOT_SUPPORTED_OR_NOT_SUBSCRIBED]
```

Data flow. Every PDU session reached the DN: 10/10 ICMP echoes from each of the seven sessions, with RTT averages of 0.72–0.93 ms. The same capture proves that the traffic used the intended user plane (Listing 2). eMBB and mIoT packets are tunnelled to UPF-A (10.10.0.1), and *both* URLLC sessions (urllc1 and multi’s second session) to UPF-B (10.10.0.3). Each session has its own TEID pair. iperf3 downlink goodput was 138 Mbit/s on eMBB, 18.7 Mbit/s on URLLC and 2.10 Mbit/s on mIoT. These match the binding cap of each slice: the tc MBR of 150 and 20 Mbit/s, and the 2 Mbit/s Session-AMBR for mIoT. HTTP requests from each slice through N6 and NAT to a public web server returned 200 in 72–80 ms.

Listing 2: GTP-U-encapsulated pings per N3 endpoint, outer→inner addresses (results/e1/gtpu_per_upf.txt; up-link rows shown).

```
10 10.10.0.2->10.10.0.1 inner 10.45.0.2->192.168.100.2 teid 0x00006eee eMBB -> UPF-A
10 10.10.0.2->10.10.0.1 inner 10.45.0.4->192.168.100.2 teid 0x0000a57f eMBB -> UPF-A (multi)
9 10.10.0.2->10.10.0.3 inner 10.46.0.2->192.168.100.2 teid 0x00000600 URLLC -> UPF-B
10 10.10.0.2->10.10.0.3 inner 10.46.0.3->192.168.100.2 teid 0x0000f718 URLLC -> UPF-B (multi)
10 10.10.0.2->10.10.0.1 inner 10.47.0.2->192.168.100.2 teid 0x000044f9 mIoT -> UPF-A
```

7.2 E2 – Signalling: registration, session establishment and slice selection

Figure 4 is the message sequence of the multi-slice UE, reconstructed directly from the capture. It shows the complete 5G registration (TS 23.502 §4.2.2.2 [11]). InitialUEMessage carries the Registration Request with the SUCI and the requested NSSAI. The AMF fetches 5G-AKA vectors via AUSF/UDM (Authentication Request/Response). The Security Mode Command then selects NIA2/NEA0. InitialContextSetupRequest carries the Registration Accept, whose **Allowed NSSAI** contains exactly the subscribed S-NSSAIs: SST 1/SD 000001 and SST 2/SD 000002 for this UE. Table 7 lists the NSSAI each UE received. The UE subscribed only to eMBB that asked for URLLC gets SST 1 as Allowed NSSAI and SST 2/SD 000002 as **Rejected NSSAI** (cause 0, “S-NSSAI not available in the current PLMN”). That is exactly the behaviour TS 24.501 specifies for a non-subscribed S-NSSAI [12].

Table 7: NSSAI in each Registration Accept, decoded from the capture (`results/e1/allowed_nssai.txt`).

UE (RAN-UE-NGAP-ID)	Allowed NSSAI (SST/SD)	Rejected NSSAI
embb1 (1), embb2 (2)	1/000001	—
urllc1 (3)	2/000002	—
miot1 (4), miot2 (5)	3/000003	—
multi (6)	1/000001, 2/000002	—
badslice (7)	1/000001	2/000002 (cause 0)

Two PDU Session Establishment Requests follow (one per S-NSSAI/DNN). The SMF then opens one PCFP session on **UPF-B** (127.0.0.17, URLLC) and one on **UPF-A** (127.0.0.7, eMBB), and each N2 PDUSessionResourceSetupRequest gives the gNB the corresponding UPF N3 address.

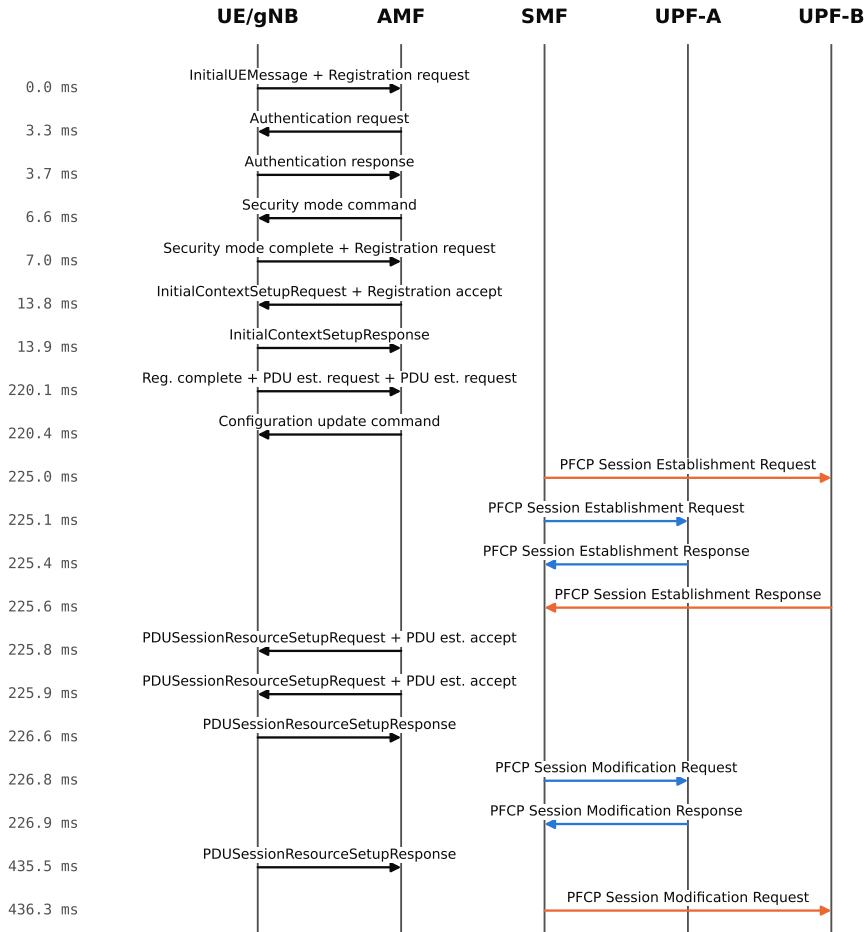


Figure 4: N2 (NGAP/NAS) and N4 (PCFP) messages for UE `imsi-001010000000006`, extracted from `results/e1/attach.pcapng`. Orange arrows: PCFP to the dedicated URLLC UPF-B; blue: shared UPF-A. Times are relative to the InitialUEMessage.

Over all UEs the capture contains 5 PCFP Session Establishments towards UPF-A and 2 towards UPF-B, which is exactly the number of eMBB+mIoT and URLLC sessions. Table 8 gives the procedure latencies. Registration completes in 7–17 ms and a PDU session in 6–15 ms. The ≈ 200 ms gap between Registration Accept and the first PDU request is UERANSIM’s own timer, not network latency. Table 9 lists the service-based operations behind these procedures (N8/N10/N12/N13 UDM/UDR/AUSF, N11 AMF→SMF, N7/N15 PCF, BSF binding). Every SBI request is counted twice, because the capture sees both hops (NF→SCP→NF).

Table 8: Procedure latency per UE (from N2 timestamps).

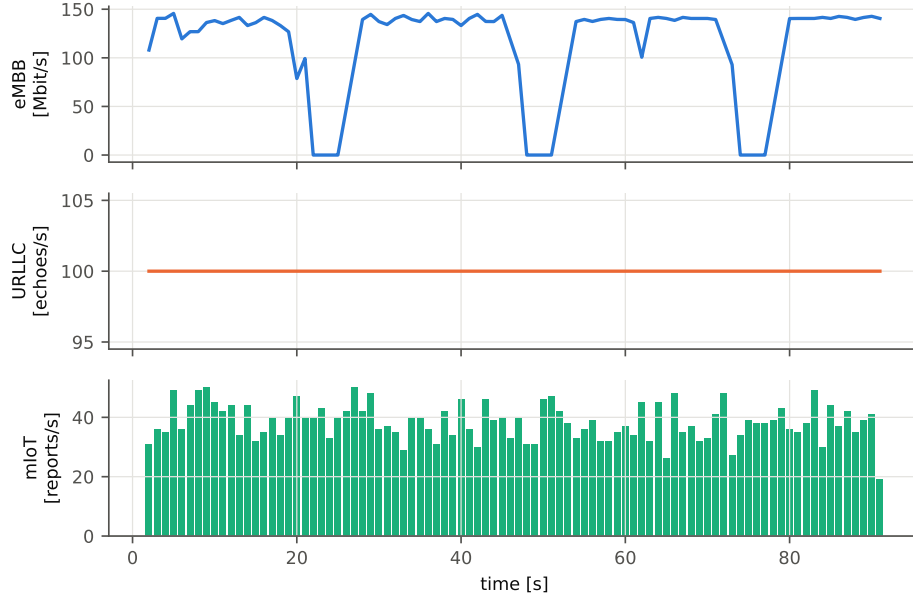
UE	SUPI	Registration [ms]	PDU sessions	PDU setup [ms]
embb1	imsi-0010100000000001	15.3	1	15.2
embb2	imsi-0010100000000002	12.1	1	9.2
urllc1	imsi-0010100000000003	15.5	1	7.9
miot1	imsi-0010100000000004	12.8	1	10.8
miot2	imsi-0010100000000005	7.3	1	10.4
multi	imsi-0010100000000006	13.8	2	6.4, 215.4
badslice	imsi-0010100000000007	17.0	0	rejected

Table 9: Most frequent SBI operations (excluding NRF heartbeats).

SBI API	Operation	Count
nudr-dr	GET /subscription-data/{supi}	56
nudr-dr	GET /policy-data/ues	28
nudm-sdm	POST /{supi}/sdm-subscriptions	28
nudr-dr	PUT /subscription-data/{supi}	27
nudm-uecm	PUT /{supi}/registrations	22
nsmf-pdusession	POST /sm-contexts	14
nsmf-pdusession	POST /sm-contexts/{id}	14
namf-comm	POST /ue-contexts/{supi}	14
nbsf-management	POST /pcfBindings	14
npcf-smpolicycontrol	POST /sm-policies	14
nudm-sdm	GET /{supi}/sm-data	14
nausf-auth	POST /ue-authentications	14

7.3 E6 – Temporal traffic profiles per slice

Figure 5 confirms that each generator reproduces its service class (scenario S1, 90 s). eMBB alternates 20 s downloads at the 150 Mbit/s slice cap (two UEs share it) with 5 s idle gaps. URLLC is a perfectly periodic 100 echoes/s at 0.05 Mbit/s. mMTC shows the Poisson-modulated arrival of 400 sensors (mean 38 reports/s for 200 devices $\times$ 2 gateways at a 10 s mean period; theory 40/s).

**Figure 5:** Per-slice offered traffic in scenario S1 (E6).

7.4 E3 – Isolation of URLLC and mMTC from an eMBB surge

Figure 6 and Table 10 show URLLC and mMTC latency before, during and after an eMBB surge of 2 UEs $\times$ 4 TCP streams. The surge is held at the eMBB slice MBR (≈ 141 Mbit/s mean). Neither slice lost a single packet, and their p99 stayed below 1.1 ms throughout the load phase. The URLLC p99 target of Table 1 (10 ms) was met in every per-second window of the load phase (worst 1.43 ms). The only violations were isolated spikes of 13–19 ms in the *idle* phases. Latency is visibly *lower* under load (p50 0.32 vs. 1.08 ms). Control experiment E7 explains this (Table 11): pure CPU busy-loops with no network traffic lower the URLLC p50 from 1.08 to 0.10 ms. The idle penalty is therefore CPU idle-state wake-up latency in the VM, not a property of the 5G stack.

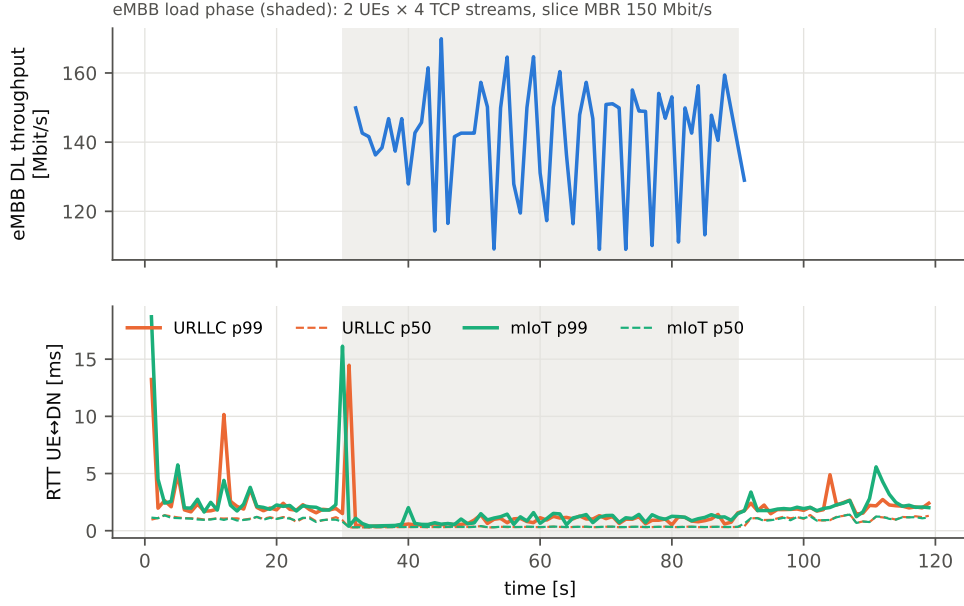


Figure 6: E3: eMBB surge (top) and URLLC/mIoT RTT (bottom). Shaded: eMBB load phase.

Table 10: E3 latency by phase (medians of per-second values).

Slice	Phase	RTT p50 [ms]	RTT p99 [ms]	worst p99 [ms]	Loss [%]
URLLC	idle (0–30 s)	1.08	1.95	13.22	0.00
URLLC	eMBB load (30–90 s)	0.32	0.86	1.43	0.00
URLLC	after (90–120 s)	1.11	2.06	4.90	0.00
mIoT	idle (0–30 s)	1.05	2.19	18.70	0.00
mIoT	eMBB load (30–90 s)	0.33	1.05	2.02	0.00
mIoT	after (90–120 s)	1.09	2.04	5.58	0.00

Table 11: E7 control experiment: URLLC RTT vs. CPU state, without any other traffic.

Condition	RTT p50 [ms]	RTT p99 [ms]
idle (0–30 s)	1.078	2.06
CPU busy-loops, no traffic (30–60 s)	0.095	2.16
idle again (60–90 s)	1.412	1.94

7.5 E4 – Live slice-parameter changes

With all three slices active, two parameters were changed live through the same API the sandbox uses (Fig. 7). At $t = 40$ s the eMBB MBR was cut from 150 to 30 Mbit/s. eMBB goodput fell from 137.6 to 28.1 Mbit/s (94 % of the new cap) within one 1-s sample, with no session re-establishment. URLLC kept exactly 100 echoes/s and mIoT ≈ 40 reports/s in every phase. At $t = 80$ s the URLLC slice received +5 ms one-way delay in each direction. URLLC p50 rose from 1.2 to 11.7 ms ($\approx +10$ ms, as configured), while the mIoT p50 stayed at 1.3 ms. Each change therefore affected only the targeted slice. The small rise of URLLC/mIoT latency after $t = 40$ s is again the CPU artefact: less eMBB traffic leaves the VM more idle.

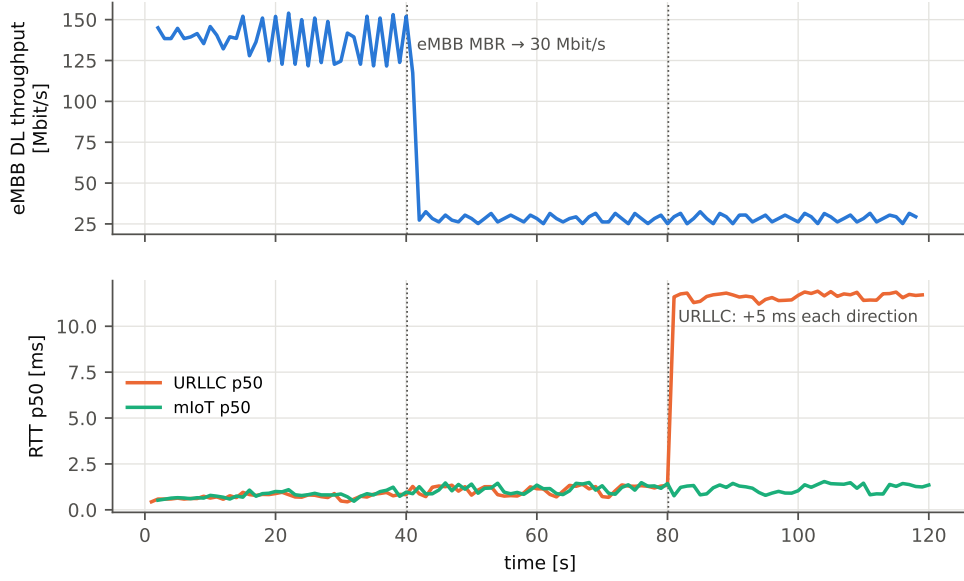


Figure 7: E4: live MBR change on eMBB at $t = 40$ s and live delay change on URLLC at $t = 80$ s (dotted lines, from the event log).

7.6 E5 – Dedicated vs. shared user plane for URLLC

To test the design decision of §2, the URLLC probe (200 Hz) ran under an identical eMBB overload in two configurations. In *dedicated* mode URLLC is anchored on UPF-B. In *shared* mode the SMF/UPF-A configs put the `urllc` DNN on UPF-A, the same UPF as eMBB. To make the offered load identical in both modes, eMBB was a fixed-rate UDP flood (2 UEs $\times$ 500 Mbit/s downlink) with Session-AMBR and the `tc` MBR lifted. The modes were alternated over three repetitions each, so that host drift affects both equally. During the load phase the VM had no idle CPU. The `iperf3` senders alone used about 3 of the 4 vCPUs, followed by `open5gs-upfd` (23–42 %) and `nr-gnb` (20–41 %). This is a deliberately extreme stress test, not an operating point.

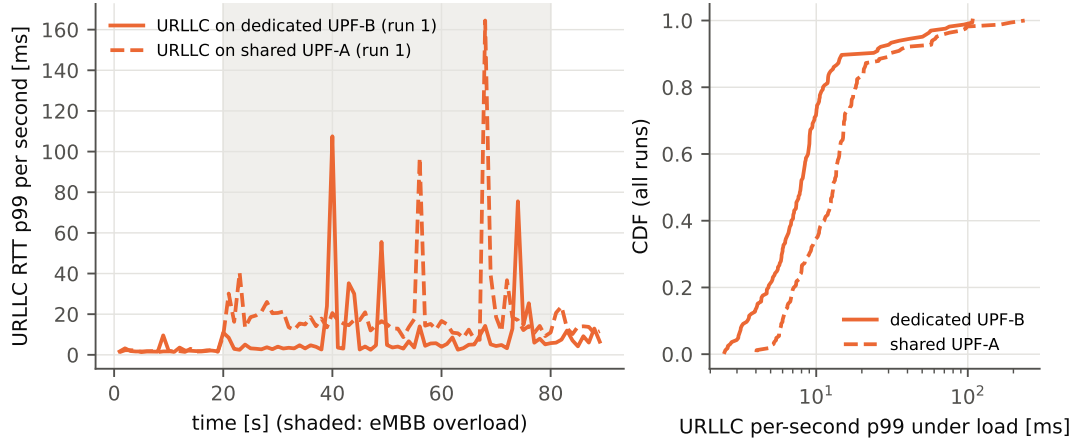


Figure 8: E5: URLLC per-second p99 RTT in the first run of each mode (left), and the CDF over all three runs per mode (right, log scale). Shaded: eMBB overload.

Table 12: E5 summary (mean $\pm$ s.d. over 3 runs per mode of the per-run median during load).

URLLC user plane	runs	load p50 [ms]	load p99 [ms]	p95 of p99 [ms]	worst p99 [ms]	loss [%]	eMBB [Mbit/s]
dedicated	3	1.50 $\pm$ 0.86	7.42 $\pm$ 2.34	43.21 $\pm$ 9.97	107.5	11.937	58
shared	3	3.77 $\pm$ 1.90	11.78 $\pm$ 4.27	59.50 $\pm$ 22.36	237.4	4.831	41

Sharing the UPF made URLLC latency worse (Fig. 8, Table 12). Pooled over all load seconds, the median of the per-second p99 rose from 7.9 ms (dedicated) to 12.9 ms (shared). The share of seconds meeting the 10 ms p99 target fell from 72 % to 35 % (Mann–Whitney $p < 10^{-11}$; the per-second samples are autocorrelated, so this overstates the significance). At the correct statistical unit (one value per run, $n = 3$ per mode), the shared configuration has a $2.5\times$ higher median RTT and a $1.6\times$ higher p99, but Welch’s test gives $p = 0.16$ and 0.22 . The direction is consistent in two of three pairs, but more repetitions would be needed to claim significance. URLLC *loss* was substantial in both modes and, unexpectedly, higher with the dedicated UPF (11.9 vs. 4.8 %). Packets are dropped in components that both modes share, the user-space gNB/UE (RLS) and the starved CPU, which UPF separation cannot isolate.

8 ANALYSIS: FROM SCENARIO TO MEASURED RESULT

Table 13 closes the loop from each scenario requirement (Table 1) through the architectural mechanism that serves it to the evidence that it works.

Table 13: Traceability: requirement $\rightarrow$ design $\rightarrow$ traffic $\rightarrow$ measured result.

Requirement	Mechanism (§2, §3)	Traffic	Evidence
Three services must be logically separated	3 S-NSSAIs, NSSF/AMF/SMF slice support, per-UE subscription	–	Allowed/Rejected NSSAI per UE (Tab. 7); unsubscribed slice refused (E1)
Each slice has its own anchor and addressing	DNN $\rightarrow$ UPF selection, per-DNN pools and tun	ICMP per session	GTP-U to 10.10.0.1 vs 10.10.0.3 (Lst. 2); PFCP $5\times$ UPF-A, $2\times$ UPF-B
UE reaches the external DN	N3/N6 namespaces, DN routes, NAT	ping, iperf3, HTTP	7/7 sessions, 0 % loss; 138/18.7/2.1 Mbit/s; HTTP 200
eMBB high rate but capped	HTB MBR + Session-AMBR	on/off TCP	goodput at 92–94 % of cap (E1, E3, E4)
URLLC p99 < 10 ms and lossless under eMBB load	dedicated UPF-B, MBR on eMBB	100 Hz probe	p99 0.86 ms, worst 1.43 ms, 0 % loss while eMBB at cap (E3)
mIoT reliable delivery	shared UPF-A, low AMBR	Poisson, 400–1000 devices	0 % loss, ACK p99 $\approx$ 1–2 ms (E3)
Operator can retune a slice live	tc class/qdisc change via sandbox	all slices	MBR and delay changes take effect within 1 s, other slices unaffected (E4)
Dedicated URLLC user plane worthwhile	UPF-B vs. shared UPF-A	UDP overload	median p99 7.9 vs. 12.9 ms; 72 % vs. 35 % of seconds within target (E5)

What the results mean for the scenario. Within its normal operating envelope (eMBB at or below its MBR), the factory network meets every requirement of Table 1. The decisive mechanism is the *rate cap on the aggressive slice*. With eMBB held at its MBR, URLLC and mIoT saw no loss and sub-2 ms p99 even when eMBB was saturating (E3). Once that cap is removed and the platform is overloaded (E5), every slice suffers, because the gNB, the RLS link and the CPU are shared. A dedicated UPF then still shifts the URLLC latency distribution down, but it does not prevent loss. The practical lesson is that slice isolation must be enforced at *every* shared resource. In this testbed those are the transport (tc) and the UPF. In a real network the RAN scheduler is the most important one, and it is precisely what a software-only RAN cannot model (§9).

Signalling cost. One registration plus one PDU session costs about 12 NGAP messages, 4 PFCP messages and about 25 distinct SBI requests per UE. The capture holds 82 NGAP frames and 354 non-NRF SBI requests for 7 UEs, the latter seen twice via the SCP. That amounts to 20–30 ms of core processing on this platform (Table 8). For a multi-slice UE the PDU procedure runs once per S-NSSAI. Both requests are handled in parallel: the two PFCP sessions on UPF-A and UPF-B are created within 0.6 ms of each other (Fig. 4). The second N2 response arrives about 200 ms later only because of UERANSIM’s internal timer.

Reading the latency numbers. The absolute RTTs (0.3–1.1 ms median) are the core and transport cost only; they include no air interface. They are also modulated by the VM’s CPU state (E7): an idle VM looks *slower* than a busy one, so we only compare measurements taken under like-for-like load. For URLLC dimensioning, the relevant results are the relative ones: the 10 ms increase produced by a 5 ms/direction transport impairment (E4), and the distribution shift caused by UPF sharing (E5).

9 LIMITATIONS OF A SOFTWARE-ONLY ENVIRONMENT

The testbed exercises real 3GPP control-plane procedures and a real GTP-U user plane, so the *functional* conclusions transfer to a hardware deployment: correct slice selection, subscription enforcement, per-slice UPF anchoring and the effect of the slice controls. The *quantitative* conclusions do not transfer directly, for the following reasons.

1. **No radio.** RLS replaces the entire NR stack below RRC/NAS. There is no PHY, no HARQ, no MAC scheduler, no numerology and no fading or interference. The air-interface latency (normally 0.5–4 ms in the RAN user plane, depending on numerology and configured/dynamic grants) and its variability are absent. Slicing *inside* the gNB (PRB partitioning, slice-aware scheduling) cannot be studied at all; our slice isolation is implemented in the core/transport only. Our sub-millisecond URLLC RTTs are therefore a lower bound on the processing cost of the core and transport, not a prediction of end-to-end URLLC latency.
2. **Shared, virtualised CPU.** Every NF, the gNB, all UEs, the traffic generators and the DN run on 4 vCPUs of one laptop SoC. Two artefacts follow and are quantified in this report. (i) CPU power management: an idle VM gives *higher* RTT than a busy one (E7: p50 1.08 ms idle vs. 0.10 ms with pure busy-loops), so “latency under load” comparisons must use the same CPU state. (ii) Scheduler noise: isolated per-second p99 spikes of 5–27 ms appear in every condition, including idle, and are caused by host or hypervisor scheduling, not by the 5G stack. We therefore compare medians and distributions over repeated runs rather than single worst cases.
3. **Throughput ceilings are CPU ceilings.** The 300–400 Mbit/s eMBB maximum reflects user-space GTP-U processing in `nr-gnb/nr-ue` and `open5gs-upfd`, not any radio capacity. Absolute throughput numbers should not be extrapolated.
4. **Slice QoS enforcement.** Open5GS’s UPF polices Session-AMBR (with up to about +15 % overshoot, `results/ambr_check.txt`), but it does not schedule by 5QI or ARP. Differentiation by 5QI is therefore only signalled, not enforced. Our tc layer adds per-slice MBR and transport delay/loss, but it is a model of transport impairment, not of RAN scheduling.
5. **Clocks and measurement.** Only RTT is measured, because one-way delay would need synchronised clocks across processes. Per-second aggregation hides sub-second dynamics, and the 1-s KPI rows mean that the sandbox reacts with up to 2 s of display lag.
6. **Scale.** Six UEs and 3000 virtual sensors behind two gateway UEs do not reproduce the RACH and signalling load of thousands of real devices. The mMTC result validates only the slice’s user plane.
7. **One measured host.** Every number comes from the macOS host. A Windows PC runs the same software on a different CPU, hypervisor (Hyper-V) and kernel (Microsoft’s WSL kernel), so its absolute latencies, throughput ceilings and CPU artefacts will differ. The Windows path is supported by the scripts. Its kernel prerequisites are checked by `wsl_setup.sh`, which was itself validated in the Linux VM, but the experiments have not yet been repeated on a Windows PC.

Validity of conclusions. The claims that *do* hold: (a) the end-to-end 5G procedures and slice selection work as specified; (b) a per-slice control (MBR, delay) is applied within one KPI interval and affects only its slice; (c) with eMBB held at its MBR, URLLC and mMTC are unaffected by an eMBB surge; (d) under platform overload, moving URLLC onto the eMBB UPF shifts its latency distribution upwards. The per-run effect is consistent in direction but not yet statistically significant with three runs per mode (§7.6). Claims about absolute latency budgets against TS 22.104 targets would need a hardware or PHY-emulating RAN (e.g. OAI/srsRAN with an SDR, or OAI’s RF simulator) and a dedicated core host.

10 CHALLENGES AND RESOLUTIONS

Table 14: Problems met during implementation and integration, in the order encountered.

Challenge	Root cause	Resolution
No Linux kernel on the Mac (SCTP, TUN, netns, tc needed)	macOS lacks these features; Docker Desktop is not installed and images are mostly amd64	Lima VM (Apple Virtualization, arm64 Ubuntu); native arm64 packages, no emulation
Homebrew refused to install Lima	Xcode licence not accepted (needs sudo)	a team member accepted the licence once; documented in the README
Provisioning script failed half-way with a syntax error open5gs-dbctl not found	the script was edited while bash was still executing it (bash reads scripts incrementally) not shipped in the Ubuntu .deb	re-ran the idempotent script; never edit a running script fetched from the matching v2.8.0 tag in 00_provision.sh
update_slice created sessions without QoS/AMBR	helper only writes the DNN	mongosh pass sets 5QI/ARP/AMBR for every session by DNN
GBR 5QI 82 risky for URLLC default flow	GBR needs GBR/MBR values and PCC rules	switched to non-GBR low-latency 5QI 80
Downlink could bypass the UPF	UE and core on the same host: replies to a local UE address are delivered locally	UEs/gNB moved into netns ran ; verified with GTP-U in the capture
URLLC RTT appeared to be 20 ms	probe only read echoes on its 10 ms send tick	select() until the next deadline; RTT timestamped on arrival
“Slice down” did not stay down	UERANSIM re-establishes configured sessions ≈ 200 ms after release	admin state + switch-off deregistration + re-attach with filtered config
No NAS details in Wireshark after Security Mode	integrity-protected NAS with NEA0 is not shown by default	nas-5gs.null_decipher:TRUE ; AMF ciphering order NEA0 first
tshark display filter rejected	Wireshark 4.2 needs commas in set syntax	pfcpl.msg_type in {50,51,52,53}
DNS failed inside UE namespace	ran netns cannot reach the VM’s stub resolver	resolve in the core netns, pass curl --resolve
eMBB throughput spikes in plots	iperf3’s trailing partial interval and duplicate samples per second	drop intervals < 0.5 s; average per UE before summing
URLLC faster under load than idle	VM CPU idle-state wake-up latency	control experiment E7; compare only like-for-like CPU states
First E5 run inconclusive	TCP eMBB load varied 270–1600 Mbit/s between runs (AMBR, TCP dynamics), confounding the comparison	fixed-rate UDP load, AMBR lifted, 3 alternating repetitions per mode
Stale UE routes after restarts	UERANSIM leaves its ip rule entries in netns ran	stop_ran.sh flushes them

Supporting both macOS and Windows. The testbed was built on a Mac, but group members use Windows PCs as well, so it was adapted to run on both. Table 15 lists the problems this raised. They were solved in the scripts, so both platforms run the same 5G configuration.

Table 15: Problems met when making the testbed run on Windows as well as macOS.

Challenge	Root cause	Resolution
Lima / <code>vz</code> not available on Windows	Apple Virtualization.framework exists only on macOS; Lima’s Windows mode only wraps WSL2	WSL2 with the same Ubuntu 24.04 and the same 4 vCPU / 6 GB (<code>vm/wslconfig.example</code>)
Different CPU architecture	Mac is arm64, most Windows PCs are amd64; the MongoDB APT line was hard-coded to arm64	<code>00_provision.sh</code> uses <code>dpkg --print-architecture</code> ; Open5GS PPA, MongoDB and UERANSIM exist for both
Scripts break after a Windows checkout	Git on Windows converts LF to CRLF, which bash cannot run	<code>.gitattributes</code> forces LF for scripts/configs; <code>wsl_setup.sh</code> detects any CRLF
MongoDB service would not start	WSL distributions run without systemd by default	<code>wsl_setup.sh</code> writes <code>systemd=true</code> to <code>/etc/wsl.conf</code> and asks for <code>wsl --shutdown</code>
Every script expects <code>/e2e5g</code>	on macOS this path is the Lima mount; WSL has no such mount	<code>wsl_setup.sh</code> links the clone to <code>/e2e5g</code> and warns if it lives under <code>/mnt/c</code> (slow, permission problems)
Unknown kernel support in WSL	SCTP, <code>sch_htb</code> and <code>sch_netem</code> depend on the WSL kernel build	live tests in <code>wsl_setup.sh</code> before installing; fallback: <code>wsl --update</code> or a Hyper-V/VirtualBox Ubuntu VM
Report screenshots tied to macOS	capture tool used the macOS Chrome path	tool picks the Chrome path per OS (<code>CHROME</code> override)
Freeing the disk afterwards	the VM and cached images take ≈ 8 GB	<code>uninstall_mac.sh</code> (macOS) and <code>wsl --unregister</code> (Windows), documented in the README

11 CONCLUSIONS AND FUTURE WORK

We built a complete, reproducible 5G SA system with three differentiated slices on a single laptop, using only open-source software. Open5GS and UERANSIM ran natively on arm64 inside a Lima VM on macOS, and the same scripts also target Windows through WSL2. Namespaces enforced a realistic RAN/core/DN separation. The scenario, architecture, traffic and measurements form one thread. The factory’s three service classes became three S-NSSAIs with their own DNN, pool, QoS profile and, for URLLC, their own UPF. The signalling shows each UE receiving exactly its subscribed Allowed NSSAI and each PDU session being anchored on the intended UPF. The KPIs show that the slice controls work and are isolated. Capping the eMBB slice was enough to keep URLLC lossless with a p99 below 1.5 ms during an eMBB surge. Under deliberate platform overload, a dedicated URLLC UPF cut the share of seconds that missed the 10 ms p99 target from 65 % to 28 %, but it could not prevent losses in the shared RAN components. The sandbox exposes all of this to a non-expert in one page.

Future work. (1) Replace RLS with a PHY-level RAN (OpenAirInterface or srsRAN Project with its RF simulator or an SDR) to study slice-aware MAC scheduling. (2) Add an O-RAN near-RT RIC (FlexRIC or O-RAN SC) with an xApp that retunes slice resources from the KPIs the sandbox already collects, closing the loop automatically. (3) Enforce QoS by 5QI with PCC rules and GBR flows for URLLC. (4) Move the NFs to Kubernetes/Helm with CPU pinning, so that the CPU artefacts of §9 can be controlled. (5) Scale mIoT with a UE simulator that supports thousands of real registrations.

REFERENCES

- [1] ITU-R. IMT vision – framework and overall objectives of the future development of IMT for 2020 and beyond. Recommendation ITU-R M.2083-0, 2015.
- [2] 3GPP. Service requirements for the 5G system. Technical Report TS 22.261, 3GPP, 2024. Release 19.
- [3] 3GPP. Service requirements for cyber-physical control applications in vertical domains. Technical Report TS 22.104, 3GPP, 2023. Release 18.
- [4] 3GPP. System architecture for the 5G system (5GS); stage 2. Technical Report TS 23.501, 3GPP, 2024. Release 18.
- [5] Lima project (CNCF). Lima: Linux virtual machines on macos. <https://lima-vm.io>, 2026. v2.2.1.
- [6] Microsoft. Windows subsystem for Linux (WSL 2). <https://learn.microsoft.com/windows/wsl/>, 2026.
- [7] Open5GS project. Open5GS: open-source 5G core and EPC. <https://open5gs.org>, 2026. v2.8.0.
- [8] Ali Güngör. UERANSIM: open-source 5G UE and RAN (gNodeB) simulator. <https://github.com/aligungr/UERANSIM>, 2025. v3.2.7.
- [9] ESnet. iperf3: network throughput measurement tool. <https://software.es.net/iperf/>. v3.16.
- [10] Snowflake Inc. Streamlit. <https://streamlit.io>. v1.65.
- [11] 3GPP. Procedures for the 5G system (5GS); stage 2. Technical Report TS 23.502, 3GPP, 2024. Release 18.
- [12] 3GPP. Non-Access-Stratum (NAS) protocol for 5G system (5GS); stage 3. Technical Report TS 24.501, 3GPP, 2024. Release 18.